LUBELSKA AKADEMIA WSEI

WYDZIAŁ TRANSPORTU I INFORMATYKI

KIERUNEK: Informatyka

SPECJALNOŚĆ: Inżynieria Oprogramowania i Technologie Mobilne

Artur Kovalov

NUMER ALBUMU: 42400

PROJEKT INŻYNIERSKI

Program do zarządzania wydatkami “Manager Wydatków”
Expense management program "Expense Manager"

Promotor: dr inż. Sylwester Korga

Lublin 2026

Spis treści

1.Wprowadzenie i zakres pracy ... 3
2. Analiza teoretyczna i technologiczna ... 4
2.1. Specyfika zarządzania kosztami w hotelarstwie ... 4
2.2. Charakterystyka wybranych technologii (State of the Art) ... 4
2.2.1. Model Single Page Application (SPA) ... 4
2.2.2 Język JavaScript (ECMAScript 6+) i DOM ... 4
2.2.3 Tailwind CSS – podejście Utility-First ... 5
2.2.4 LocalStorage jako magazyn danych ... 5
2.3. Środowisko programistyczne i narzędzia ... 5
3. Analiza i projekt systemu ... 6
3.1. Wymagania funkcjonalne i niefunkcjonalne ... 6
3.2. Projekt struktury danych ... 7
3.3. Projekt interfejsu (UI/UX) ... 8
4. Implementacja systemu ... 9
4.1. Struktura plików i inicjalizacja ... 9
4.2. Implementacja kluczowych modułów ... 9
5. Testowanie i weryfikacja ... 11
5.1. Metodyka testów ... 11
5.2. Wyniki testów funkcjonalnych ... 11
5.3. Analiza wydajności ... 12
6. Podsumowanie i wnioski ... 13
6.1. Ocena stopnia realizacji celów ... 13
6.2. Wnioski końcowe ... 13
6.3. Kierunki dalszego rozwoju ... 13
Bibliografia ... 15
Spis Rysunków ... 16
Spis Tabel ... 17
Załączniki ... 18

1.Wprowadzenie i zakres pracy

Temat: Program do zarządzania wydatkami “Manager Wydatków” Expense management program "Expense Manager"

Niniejsza praca ma na celu zaprojektowanie oraz implementacji autorskiego systemu informatycznego wspierającego zarządzanie kosztami operacyjnymi w obiektach hotelarskich. Celem utylitarnym jest wsparcie małych i średnich obiektów HoReCa w zarządzaniu finansami.

Zakres niniejszej pracy inżynierskiej obejmuje kompleksowy proces wytwarzania dedykowanego oprogramowania – od analizy teoretycznej problemu biznesowego, poprzez projektowanie i implementację, aż po weryfikację gotowego produktu. Treść pracy została podzielona na sześć merytorycznych rozdziałów, które logicznie prowadzą przez poszczególne etapy realizacji projektu. W pierwszym rozdziale przedstawiono wprowadzenie do tematyki, zdefiniowano główny cel badawczy oraz uzasadniono wybór tematu, wskazując na istotną lukę w dostępności narzędzi IT dla małych obiektów hotelarskich. W drugim rozdziale skupiono się na fundamencie teoretycznym i technologicznym. Przedstawiono w nim podstawy prowadzenia działalności operacyjnej w branży HoReCa oraz specyfikę controllingu kosztów. Dokonano również charakterystyki wykorzystanych technologii webowych, uzasadniając wybór architektury Single Page Application (SPA) oraz mechanizmu LocalStorage. Trzeci rozdział poświęcono fazie analityczno-projektowej. Zdefiniowano w nim szczegółowe wymagania funkcjonalne i niefunkcjonalne stawiane systemowi, opracowano model struktury danych oraz zaprojektowano interfejs użytkownika i logikę interakcji.W czwartym rozdziale opisano proces implementacji systemu „Manager Wydatków”. Przedstawiono w nim kluczowe rozwiązania programistyczne, w tym algorytmy obsługi dynamicznych formularzy faktur, mechanizmy agregacji danych finansowych oraz sposób generowania raportów. Piąty rozdział zawiera opis metodyki weryfikacji oprogramowania. Zaprezentowano w nim scenariusze testowe oraz wyniki przeprowadzonych testów funkcjonalnych i wydajnościowych, które potwierdzają stabilność i poprawność działania aplikacji. W szóstym rozdziale dokonano syntezy wyników pracy, oceniono stopień realizacji założonych celów badawczych i inżynierskich oraz wskazano potencjalne kierunki dalszego rozwoju systemu.

2. Analiza teoretyczna i technologiczna

2.1. Specyfika zarządzania kosztami w hotelarstwie

Działalność hotelarska charakteryzuje się wysokim udziałem kosztów stałych oraz dużą zmiennością popytu sezonowego. Kluczowym wskaźnikiem efektywności w gastronomii hotelowej (śniadania, bar) jest *Food Cost* (FC), definiowany wzorem:

$$FC = \frac{Koszt\ zużytych\ surowców}{Przychód\ ze\ sprzedaży} * 100\%$$

W przypadku śniadań wliczonych w cenę noclegu, analizuje się tzw. "Koszt talerza" (koszt surowca na jedną osobę). Tradycyjne metody obliczania FC (inwentaryzacja miesięczna) są zbyt powolne w warunkach dynamicznych zmian cen rynkowych. Istnieje zapotrzebowanie na narzędzia typu *Real-Time BI*, które pozwalają monitorować ten wskaźnik na bieżąco, po wprowadzeniu każdej faktury zakupowej.

2.2. Charakterystyka wybranych technologii (State of the Art)

2.2.1. Model Single Page Application (SPA)

W projekcie zdecydowano się na architekturę SPA. W przeciwieństwie do klasycznych stron MPA (Multi Page Application), SPA ładuje tylko jeden dokument HTML, a cała zawartość jest dynamicznie podmieniana przez skrypty uruchamiane w przeglądarce. Gwarantuje to płynność działania zbliżoną do aplikacji desktopowych oraz redukcję transferu danych sieciowych.

Tabela 1. Porównanie architektury SPA i MPA

Cecha	Single Page Application (SPA)	Multi Page Application (MPA)
Szybkość działania	Wysoka (po pierwszym załadowaniu)	Średnia (każda strona ładuje się od nowa)
Obciążenie serwera	Niskie (tylko API/JSON)	Wysokie (generowanie HTML na serwerze)
Doświadczenie użytkownika	Płynne, "aplikacyjne"	Przeładowania strony, "mruganie"
SEO (Pozycjonowanie)	Trudniejsze (wymaga SSR)	Łatwe (naturalne dla wyszukiwarek)
Zastosowanie w projekcie	**TAK (Wybrano)**	NIE

2.2.2 Język JavaScript (ECMAScript 6+) i DOM

Język JavaScript stanowi fundament warstwy logicznej aplikacji. Wykorzystano standard ES6+, w tym nowoczesne funkcje takie jak Arrow Functions, destrukturyzację obiektów oraz

Template Literals. Manipulacja drzewem DOM (Document Object Model) odbywa się natywnie, co zapewnia wyższą wydajność niż przy użyciu starszych bibliotek typu jQuery.

2.2.3 Tailwind CSS – podejście Utility-First

Do warstwy prezentacji wybrano framework Tailwind CSS. Pozwala on na szybkie tworzenie responsywnych interfejsów (RWD) poprzez stosowanie klas użytkowych bezpośrednio w kodzie HTML (np. flex, p-4, text-center). Przyspiesza to proces prototypowania i zapewnia spójność wizualną projektu bez konieczności pisania rozległych arkuszy stylów.

2.2.4 LocalStorage jako magazyn danych

Ze względu na wymagania dotyczące prywatności i eliminacji kosztów serwera, zastosowano Web Storage API (localStorage). Jest to baza danych typu klucz-wartość wbudowana w przeglądarkę, umożliwiająca przechowywanie danych tekstowych (JSON). Dane te nie wygasają po zamknięciu przeglądarki, co pozwala na trwałe przechowywanie historii operacji na urządzeniu użytkownika.

2.3. Środowisko programistyczne i narzędzia

Projekt realizowano przy użyciu edytora kodu Visual Studio Code, wyposażonego w rozszerzenia wspomagające formatowanie (Prettier) oraz podpowiadanie składni. Do kontroli wersji kodu wykorzystano system Git, co umożliwiło bezpieczne wprowadzanie zmian i eksperymentowanie z nowymi funkcjami na osobnych gałęziach (branches). Debugowanie i analiza wydajności odbywały się przy użyciu narzędzi deweloperskich przeglądarki Chrome (Chrome DevTools).

3. Analiza i projekt systemu

3.1. Wymagania funkcjonalne i niefunkcjonalne

Na podstawie analizy potrzeb użytkownika końcowego, zdefiniowano kluczowe wymagania systemu. Wymagania funkcjonalne:

Tabela 2. Zestawienie wymagań funkcjonalnych systemu

ID Wymagania	Nazwa modułu	Opis funkcjonalności	Priorytet
F-01	Baza Produktów	Możliwość dodawania, edycji i usuwania produktów oraz przypisywania im kategorii i jednostek miary.	Wysoki
F-02	Rejestr Zakupów	Formularz umożliwiający wprowadzenie faktury zawierającej wiele pozycji produktowych jednocześnie.	Wysoki
F-03	Ewidencja Operacyjna	Rejestracja dziennej liczby śniadań oraz liczby gości hotelowych (obłożenia).	Średni
F-04	Analityka (Dashboard)	Automatyczne wyliczanie wskaźników KPI (Food Cost) i prezentacja na wykresach.	Wysoki
F-05	Eksport Danych	Generowanie raportów w formacie PDF oraz zrzut bazy do pliku JSON (Backup).	Niski

Wymagania niefunkcjonalne:

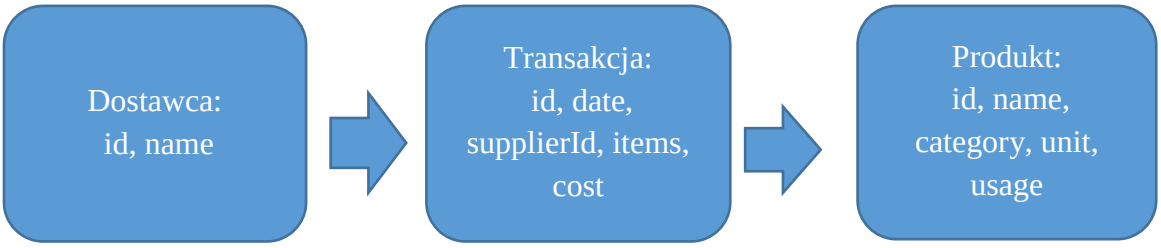
Tabela 3. Zestawienie wymagań niefunkcjonalnych systemu

ID	Kategoria	Opis wymagania	Priorytet
NF-01	**Dostępność (Offline)**	Aplikacja musi zachować pełną funkcjonalność (dodawanie faktur, przeglądanie wykresów) bez dostępu do połączenia internetowego po wstępnym załadowaniu zasobów.	**Krytyczny**
NF-02	**Wydajność**	Czas ładowania aplikacji (Time to Interactive) nie może przekraczać 2 sekund. Czas generowania raportu PDF dla 1000 transakcji nie może przekroczyć 1 sekundy.	**Wysoki**
NF-03	**Responsywność (RWD)**	Interfejs użytkownika musi automatycznie dostosowywać układ elementów do	**Wysoki**

		rozdzielczości ekranu, zapewniając czytelność na urządzeniach mobilnych (min. szerokość 360px) oraz tabletach.	
NF-04	**Prywatność Danych**	System musi działać w architekturze *Local-First*. Dane finansowe są przechowywane wyłącznie w pamięci przeglądarki użytkownika (LocalStorage) i nie są przesyłane na serwery zewnętrzne.	**Krytyczny**
NF-05	**Kompatybilność**	Aplikacja musi działać poprawnie na najnowszych wersjach popularnych przeglądarek internetowych: Google Chrome, Microsoft Edge, Mozilla Firefox.	**Średni**
NF-06	**Pojemność**	System musi umożliwiać przechowywanie historii transakcji z okresu minimum 2 lat działania małego obiektu (ok. 10 000 rekordów JSON) w ramach limitu pamięci przeglądarki.	**Średni**

3.2. Projekt struktury danych

System operuje na nierelacyjnym modelu danych opartym na obiektach JSON.



Rys. 1. Model danych aplikacji przedstawiający relacje między obiektami Transaction, Product i Supplier.

Główne struktury danych zaprojektowane w systemie:

- Produkty (products): Tablica obiektów zawierająca unikalne ID, nazwę, kategorię (np. "Nabiał") i jednostkę miary.
- Transakcje (transactions): Tablica przechowująca historię zakupów. Każdy obiekt zawiera datę, ID dostawcy, zagnieżdżoną tablicę zakupionych pozycji oraz kwotę łączną.

3.3. Projekt interfejsu (UI/UX)

Interfejs zaprojektowano zgodnie z zasadami *Material Design* i *Mobile First*. Główny widok (Dashboard) zawiera karty KPI (Key Performance Indicators) oraz wykresy, natomiast operacje wprowadzania danych odbywają się w oknach modalnych (pop-up), aby użytkownik nie tracił kontekstu pracy. Zastosowano kontrastową kolorystykę dla przycisków akcji (Call to Action).

4. Implementacja systemu

4.1. Struktura plików i inicjalizacja

Aplikacja składa się z głównego pliku HTML, który importuje biblioteki zewnętrzne (Tailwind, Chart.js, Lucide Icons) poprzez sieć CDN. Inicjalizacja odbywa się w momencie zdarzenia window.onload, gdzie następuje pobranie danych z localStorage i pierwsze renderowanie widoku.

4.2. Implementacja kluczowych modułów

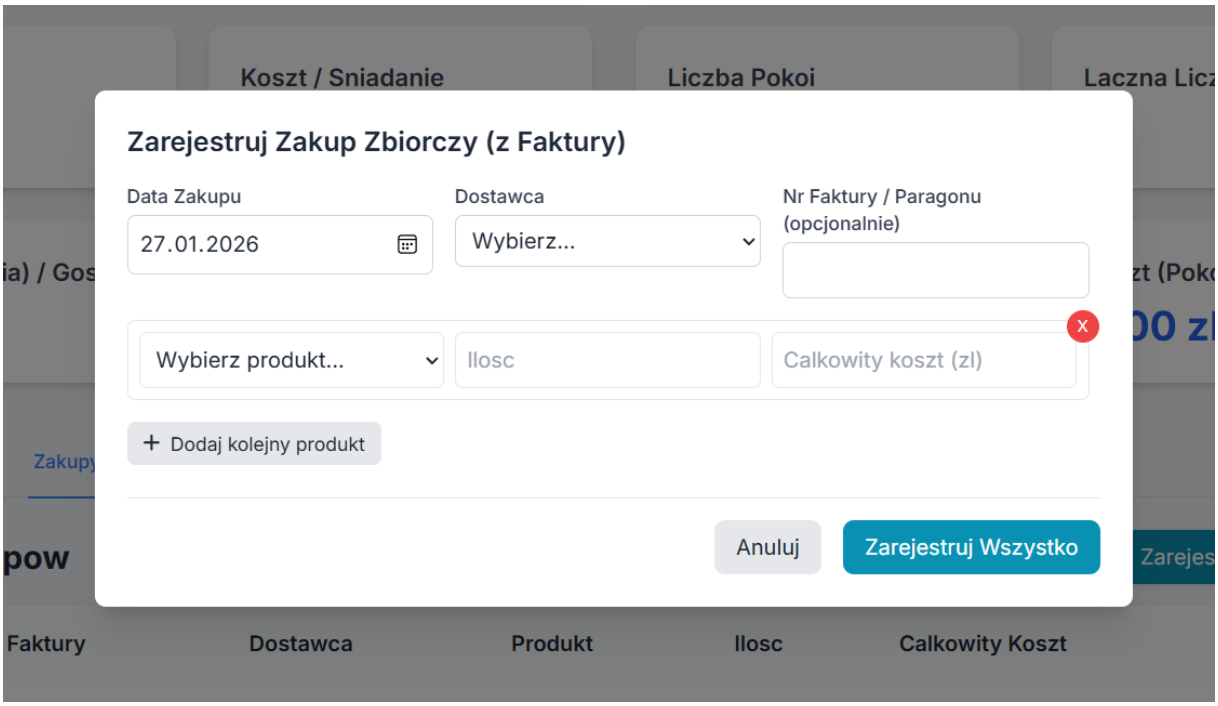
Moduł dynamicznego formularza faktury

Najbardziej złożonym elementem systemu jest obsługa faktur zbiorczych. Implementacja opiera się na dynamicznym tworzeniu elementów DOM w pętli.

Listing 1. Funkcja dodająca wiersz produktu do faktury (JavaScript)

```
function addInvoiceRow() {
  const container = document.getElementById('items-container');
  const rowId = Date.now(); // Generowanie unikalnego ID

  // Tworzenie struktury HTML dla nowego wiersza
  const rowHTML = `
    <div class="flex gap-2 mb-2" id="row-${rowId}">
      <select class="border p-2 rounded w-full product-select">
        ${generateProductOptions()}
      </select>
      <input type="number" class="border p-2 w-24 price" placeholder="Cena">
      <input type="number" class="border p-2 w-20 qty" placeholder="Ilość">
      <button onclick="removeRow(${rowId})" class="text-red-500">Usuń</button>
    </div>
  `;
  container.insertAdjacentHTML('beforeend', rowHTML);
}
```



Rys. 2. Widok modalnego okna dodawania transakcji z dynamiczną listą pozycji.

Algorytmy analityczne i wizualizacja

Do wizualizacji danych wykorzystano bibliotekę Chart.js. Dane przed wyświetleniem są agregowane metodą reduce, która grupuje wydatki według kategorii.

Listing 2. Agregacja kosztów wg kategorii

```
const categoryData = transactions.reduce((acc, curr) => {
   // Sumowanie kwot dla danej kategorii
   acc[curr.category] = (acc[curr.category] || 0) + curr.amount;
   return acc;
}, {});
```

Rys. 3. Panel analityczny prezentujący strukturę kosztów wg kategorii.

Mechanizm eksportu danych

System umożliwia generowanie raportów PDF przy użyciu biblioteki jsPDF. Kluczowym wyzwaniem inżynierskim była obsługa polskich znaków diakrytycznych. Rozwiązano to poprzez implementację niestandardowej czcionki zakodowanej w formacie Base64, która jest ładowana do wirtualnego systemu plików biblioteki przed wygenerowaniem dokumentu.

5. Testowanie i weryfikacja

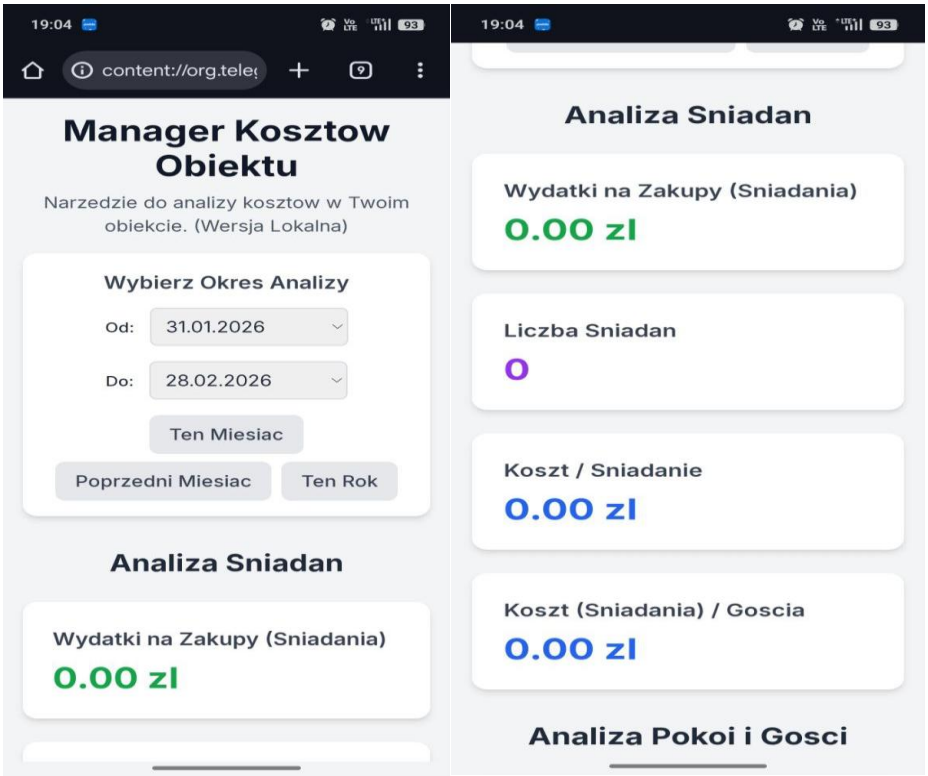
5.1. Metodyka testów

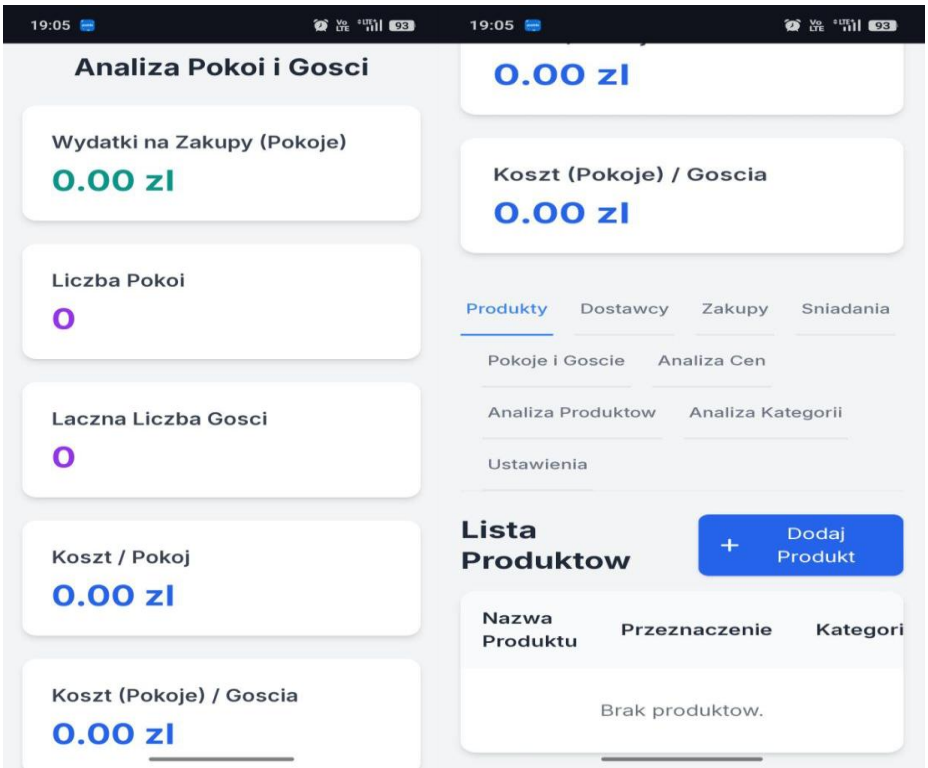
W celu weryfikacji oprogramowania przeprowadzono testy systemowe "czarnoskrzynkowe" (Black-box testing). Skupiono się na poprawności obliczeń matematycznych, integralności danych przy operacjach CRUD oraz odporności na błędy użytkownika (walidacja danych wejściowych).

5.2. Wyniki testów funkcjonalnych

Tabela 4. Scenariusze testowe i wyniki

ID	Przypadek testowy	Warunki wstępne	Oczekiwany rezultat	Wynik
T1	Dodanie produktu do bazy	Pusta lista produktów	Produkt pojawia się na liście wyboru w formularzu	POZYTYWNY
T2	Obliczenie sumy faktury	Dodano 2 pozycje po 10 PLN	Suma całkowita automatycznie wynosi 20.00 PLN	POZYTYWNY
T3	Walidacja ceny ujemnej	Wpisano "-50" w polu ceny	System blokuje zapis lub wyświetla błąd walidacji	POZYTYWNY
T4	Responsywność RWD	Ekran o szerokości 360px	Elementy nie wychodzą poza ekran, menu boczne zwija się	POZYTYWNY
T5	Eksport danych	Istniejące transakcje	Pobranie pliku .pdf z poprawnymi danymi	POZYTYWNY





Rys. 4. Prezentacja interfejsu aplikacji na urządzeniu mobilnym.

5.3. Analiza wydajności

Przeprowadzono testy obciążeniowe polegające na wprowadzeniu do bazy 1000 transakcji. Czas generowania wykresów oraz eksportu PDF nie przekroczył 500ms, co potwierdza wysoką wydajność zastosowanej architektury Client-Side.

6. Podsumowanie i wnioski

6.1. Ocena stopnia realizacji celów

Zrealizowana praca inżynierska doprowadziła do powstania w pełni funkcjonalnego systemu „Manager Wydatków”. Wszystkie założone cele, w tym implementacja ewidencji faktur, bazy produktów oraz automatyzacja wskaźników KPI, zostały osiągnięte. Aplikacja cechuje się wysoką wydajnością oraz intuicyjnym interfejsem użytkownika.

6.2. Wnioski końcowe

Analiza procesu tworzenia i testowania aplikacji pozwala na sformułowanie następujących wniosków:

- Zastosowanie architektury SPA bez backendu znacząco obniża barierę wejścia i koszty wdrożenia systemu w małych firmach.
- Technologia LocalStorage jest skutecznym rozwiązaniem dla aplikacji niewymagających centralizacji danych, zapewniając jednocześnie wysoką prywatność.
- Wykorzystanie nowoczesnych frameworków (Tailwind CSS) skraca czas wytwarzania oprogramowania, umożliwiając szybkie dostosowanie produktu do potrzeb klienta.

6.3. Kierunki dalszego rozwoju

Zrealizowany w ramach niniejszej pracy system „Manager Wydatków” w wersji 1.0 stanowi w pełni funkcjonalne narzędzie typu MVP (*Minimum Viable Product*). Architektura aplikacji została jednak zaprojektowana w sposób modułowy, co umożliwia jej łatwą rozbudowę w przyszłości. Na podstawie analizy obecnych trendów w branży IT oraz potrzeb rynku hotelarskiego, zidentyfikowano cztery główne obszary dalszego rozwoju oprogramowania.

1. Transformacja w Progressive Web App (PWA) Obecna wersja aplikacji działa w przeglądarce internetowej. Naturalnym krokiem rozwojowym jest pełne wdrożenie standardu PWA. Pozwoliłoby to na:

- Instalację aplikacji na ekranie głównym urządzenia mobilnego (smartfona/tabletu) bez konieczności publikowania jej w sklepach App Store czy Google Play.
- Wykorzystanie mechanizmu *Service Workers* do jeszcze lepszego zarządzania pracą w trybie offline i cache'owania zasobów.
- Wysyłanie powiadomień *Push* (np. przypomnienie o konieczności zamówienia towaru, gdy stany magazynowe spadną poniżej minimum).

2. Synchronizacja z chmurą (Cloud Sync) Aby wyeliminować główne ograniczenie technologii LocalStorage (brak dostępności danych na innych urządzeniach), planowane jest wdrożenie opcjonalnego modułu synchronizacji. Rozważane jest wykorzystanie Google Drive API lub usługi Firebase Realtime Database. Dzięki temu użytkownik mógłby wprowadzać dane na tablecie w magazynie, a następnie analizować je na komputerze w biurze, przy zachowaniu pełnej spójności danych.

3. Automatyzacja wprowadzania danych (OCR i AI) Wprowadzanie faktur, choć usprawnione przez formularz zakupu zbiorczego, nadal wymaga pracy manualnej. Kierunkiem rozwoju jest integracja z bibliotekami OCR (*Optical Character Recognition*), takimi jak Tesseract.js. Umożliwiłoby to zrobienie zdjęcia papierowej fakturze, a system automatycznie rozpoznałby pozycje, ceny i nazwy produktów, prosząc użytkownika jedynie o weryfikację.

4. Moduł prognozowania kosztów Wykorzystując zgromadzone dane historyczne, system mógłby zostać rozbudowany o proste algorytmy uczenia maszynowego (Machine Learning). Pozwoliłoby to na prognozowanie przyszłych wydatków w oparciu o sezonowość (np. przewidywanie wzrostu zużycia produktów śniadaniowych w sezonie letnim) oraz ostrzeganie menedżera przed potencjalnym przekroczeniem budżetu jeszcze przed dokonaniem zakupów.

Bibliografia

Literatura zwarta i artykuły:

1. Crockford D., *JavaScript: Elementy dobrego stylu*, Wydawnictwo Helion, Gliwice 2009.

2. Flanagan D., *JavaScript. Przewodnik. Wydanie VII*, Wydawnictwo Helion, Gliwice 2021.

3. Freeman A., *Podręcznik front-end developera. HTML5, CSS3 i JavaScript*, Wydawnictwo Helion, Gliwice 2019.

4. Mikulski T., *Rachunkowość zarządcza w gastronomii*, PWE, Warszawa 2018. (Odniesienie do zagadnień Food Cost i controllingu).

5. Resig J., Bibeault B., Maras J., *Ninja w JavaScript. Wydanie II*, Wydawnictwo Helion, Gliwice 2017.

6. Simpson K., *Tajniki języka JavaScript. Na drodze do biegłości*, Wydawnictwo Helion, Gliwice 2016.

Dokumentacja techniczna i źródła internetowe (Netografia):

7. *Chart.js Documentation*, https://www.chartjs.org/docs/latest/ [dostęp: 04.02.2026].

8. *ECMAScript® 2025 Language Specification*, https://tc39.es/ecma262/ [dostęp: 04.02.2026].

9. *Git – Dokumentacja systemu kontroli wersji*, https://git-scm.com/doc [dostęp: 04.02.2026].

10. *jsPDF Library Documentation*, https://artskydj.github.io/jsPDF/docs/jsPDF.html [dostęp: 04.02.2026].

11. *MDN Web Docs – LocalStorage (Web Storage API)*, Mozilla Developer Network, https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage [dostęp: 04.02.2026].

12. *MDN Web Docs – Single Page Application (SPA)*, Mozilla Developer Network, https://developer.mozilla.org/en-US/docs/Glossary/SPA [dostęp: 04.02.2026].

13. *Tailwind CSS Documentation – Utility-First Fundamentals*, https://tailwindcss.com/docs/utility-first [dostęp: 04.02.2026].

14. *Visual Studio Code Documentation*, https://code.visualstudio.com/docs [dostęp: 04.02.2026].

Spis Rysunków

- **Rys. 1.** Model danych aplikacji przedstawiający relacje między obiektami Transaction, Product i Supplier.
- **Rys. 2.** Widok modalnego okna dodawania transakcji z dynamiczną listą pozycji.
- **Rys. 3.** Panel analityczny prezentujący strukturę kosztów.
- **Rys. 4.** Prezentacja interfejsu aplikacji na urządzeniu mobilnym.

Spis Tabel

- **Tabela 1.** Porównanie architektury SPA i MPA
- **Tabela 2.** Zestawienie wymagań funkcjonalnych systemu
- **Tabela 3.** Zestawienie wymagań niefunkcjonalnych systemu
- **Tabela 4.** Scenariusze testowe i wyniki

Załączniki

Załącznik 1. Kluczowe fragmenty kodu źródłowego aplikacji

Listing 1. Wrapper do obsługi LocalStorage (Obiekt DB) Poniższy fragment kodu prezentuje warstwę abstrakcji utworzoną w celu łatwego zapisu i odczytu danych w przeglądarce, co realizuje wymaganie architektury Local-First.

```
const DB = {
   get: (key, defaultValue = '[]') => JSON.parse(localStorage.getItem(key) || defaultValue),
   set: (key, data) => localStorage.setItem(key, JSON.stringify(data)),
   remove: (key) => localStorage.removeItem(key),
};
```

Listing 2. Algorytm filtrowania i agregacji danych (Business Logic) Funkcja odpowiedzialna za pobieranie transakcji, śniadań oraz danych o pokojach w wybranym przez użytkownika zakresie dat. Jest to kluczowy element modułu analitycznego.

```
const getDataForRange = () => {
   const rangedPurchases = purchases.filter(p => {
      const d = new Date(p.date);
      return d >= startDate && d <= endDate;
   });

   const rangedBreakfastsCount = breakfasts
      .filter(b => {
         const d = new Date(b.date);
         return d >= startDate && d <= endDate;
      })
      .reduce((sum, b) => sum + b.count, 0);

   const rangedRooms = cleanedRooms.filter(r => {
      const d = new Date(r.date);
      return d >= startDate && d <= endDate;
   });

   const rangedRoomsCount = rangedRooms.reduce((sum, r) => sum + (r.roomCount || 0), 0);
   const rangedGuestsCount = rangedRooms.reduce((sum, r) => sum + (r.guestCount || 0), 0);

   return { rangedPurchases, rangedBreakfastsCount, rangedRoomsCount,
rangedGuestsCount };
};
```

Listing 3. Obsługa dynamicznego formularza zakupów zbiorczych Implementacja dodawania dynamicznych wierszy w formularzu faktury, umożliwiająca wprowadzenie wielu produktów jednocześnie.

```
const createBatchItemRow = () => {
   const div = document.createElement('div');
   div.className = 'batch-item grid grid-cols-1 md:grid-cols-3 gap-2 border p-2 rounded-md
relative';
```

```
    const selectOptions = '<option value="">Wybierz produkt...</option>' +
        [...products].sort((a,b) => a.name.localeCompare(b.name))
        .map(p => `<option value="${p.id}">${p.name}</option>`).join('');

    div.innerHTML = `
        <select class="batch-product-select p-2 border rounded-md w-
full">${selectOptions}</select>
        <input type="number" step="0.01" placeholder="Ilosc" class="batch-quantity p-2 border
rounded-md w-full">
        <input type="number" step="0.01" placeholder="Calkowity koszt (zl)" class="batch-cost
p-2 border rounded-md w-full">
        <button type="button" class="bg-red-500 text-white rounded-full h-6 w-6 flex items-
center justify-center text-xs" onclick="this.parentElement.remove()">X</button>
    `;
    batchItemsContainer.appendChild(div);
};
```

Załącznik 2. Struktura pliku JSON (Format zapisu danych)
Poniżej przedstawiono strukturę obiektu JSON wykorzystywanego do tworzenia kopii zapasowej (Backup) oraz przechowywania danych w LocalStorage.

```
{
  "products": [
    {
      "id": "uuid-v4-string",
      "name": "Mleko 3.2%",
      "unit": "l",
      "type": "volume",
      "category": "Nabiał",
      "productUsage": "food"
    }
  ],
  "transactions": [
    {
      "id": "uuid-v4-string",
      "date": "2026-02-04T12:00:00.000Z",
      "supplierId": "uuid-supplier",
      "items": [
        {
          "productId": "uuid-product",
          "quantity": 10,
          "cost": 35.50
        }
      ],
      "totalCost": 35.50
    }
  ]
}
```

Załącznik 3. Opis zawartości nośnika cyfrowego

Do pracy dołączono płytę CD / pendrive zawierający:

1. **Katalog `Dokumentacja`**:
    a. Elektroniczna wersja niniejszej pracy inżynierskiej (PDF, DOCX).
    b. Prezentacja multimedialna projektu.
2. **Katalog `Aplikacja`**:
    a. Plik `index.html` – Główny plik uruchomieniowy systemu (Single Page Application).
    b. Plik `README.txt` – Krótka instrukcja uruchomienia (wymagana przeglądarka internetowa z obsługą JavaScript ES6+).
3. **Katalog `Source`**:
    a. Pełny kod źródłowy aplikacji (HTML, CSS, JS).